National University of Computer and Emerging Sciences

Fast School of Computing Fall 2026

CS-1002 – PF (CS Department)

Assignment 2

Section: *All sections*

Submission Deadline: 11:59 PM. October 18, 2026. Sunday

Submission Instructions:

1. Assignments are to be done individually. You must complete this assignment by yourself. You cannot work with anyone else in the class or with someone outside of the class for this assignment. The code you write must be your own, and you must understand each part of it. You are encouraged, however, to get help from the instructional staff through Google Classroom.
2. Plagiarism: Plagiarism of any kind (copying from others, copying from the internet or using AI, etc.) is not allowed. If found plagiarised, you will be awarded zero marks in the whole assignment category. Repeating such an act can lead to strict disciplinary actions and failure in the course. Do note that AI and plagiarism will be thoroughly checked across the batch and different LLMs. If any hint of similarity is found, strict action will be taken.
3. A demo for this assignment will be held, where plagiarism and solutions to the assignment's questions will be assessed, and marks for the assignment will be given accordingly.
4. Please start early; otherwise, you will struggle with the assignment.
5. Keep an eye out for the comments on the assignment. Sometimes queries are asked in the comments, which add constraints to the questions of the assignment.
6. All queries must be asked only in the Google Classroom comment section for the assignment, allowing others with similar questions to be able to have the answers to their questions.
7. If you have a query, kindly check if anyone has already asked a similar query beforehand.
8. The assignment should be submitted as one single .cpp file.
9. Failure to do so will lead to nullification of the whole assignment.
10. The file that needs to be submitted will be using the following naming convention: ROLLNUMBER_SECTION.cpp (e.g. 25i-0001_A.cpp).
11. A submission portal will be uploaded for you to submit the assignment.
12. Submit the .cpp file on the portal only within the deadline. Submissions on emails will not be accepted.
13. Try to submit at-least 1 hour before the deadline to tackle potential internet issues regarding the submission.
14. Submissions other than the portal(e.g. email, GCR, etc.) will not be accepted.
15. The entire assignment must be done in **one single switch** statement only. All questions and all parts of each question must run from that singular switch.
16. Remember only main() is a menu-driven switch with one case per question. Inside each case, follow that question's own restrictions.
17. The student is solely responsible for checking the final submission for issues like corrupt files sent. If we cannot download the file for any reason, it will lead to zero marks in the assignment

Note: Follow the given instructions; failing to do so may result in a zero in this assignment.

Question 1 : The Reckoning [15 Marks]

The speakers crackle: "Players, the next game begins shortly." The pink-suited guards line the walls, the VIPs are placing their bets, and the dormitory shop is open for the last time. You have a limited stock of cash, a debt hanging over you, and one chance to walk out with the prize.

Your task: compute the final bill, how much your cash covers, whether you survive the next game, and what your balance looks like at the end.

All conditional logic must use ternary operators only.
Use integer arithmetic only throughout.

Input Order

- type {'A','T','G'}: Athlete, Thinker, Gambler
- game {1..5}: 1 = Red Light Green Light, 2 = Honeycomb, 3 = Tug of War, 4 = Marbles, 5 = Glass Bridge
- timeLeft [0..120]: minutes until the game starts
- guardAlert [0..100]
- injuries [0..3]
- lightsOn {0,1}: 1 = dorm lights are on
- cameras {0,1}: 1 = VIP cameras active
- takeNeedles {0,1}
- needles [0..40]
- takeBandages {0,1}
- bandages [0..30]
- takeMarbles {0,1}
- marbles [0..100]
- takeRope {0,1}
- ropeM [0..60]: meters of rope
- packKg [0..150]: weight you carry
- stamina [0..100]
- playersLeft [2..456]
- bet {0,1,2}: 0 = No bet, 1 = SAFE, 2 = RISKY
- stake [0..2000]: ignored if bet == 0
- favor {0,1,2,3}: 0 = None, 1 = INSIDER, 2 = HUSH, 3 = MEDPASS
- cash [0..6000]
- debt [0..3000]
- card {0,1}: 1 = you hold your invitation card
- ally {0,1}: 1 = you have an ally

Base Unit Prices

- Needle: 150 each
- Bandage: 350 each
- Marble: 60 each
- Rope: 120 per meter

Type Perks (replace the unit price, apply before tiers)

- Athlete: Rope costs 100 per meter
- Thinker: Needle costs 120 each
- Gambler: Marble costs 45 each

Item Costs (before surcharge)
Each cost is 0 if its take flag is 0.

- NeedleCost: the first 10 needles at full unit price, every needle beyond 10 at (unit * 75) / 100
- MarbleCost: the first 50 marbles at full unit price, every marble beyond 50 at (unit * 50) / 100
- BandageCost: buy 2 get 1 free, so you pay for (bandages − bandages / 3) bandages
- RopeCost: ropeM * unit price
- (For NeedleCost and MarbleCost, compute the discounted unit price first, then multiply by the number of discounted items.)

Alert Adjustment
Start alertAdj = guardAlert, then:

- add marbles / 5 if takeMarbles == 1
- add ropeM / 3 if takeRope == 1
- add 10 if lightsOn == 1
- add 5 if cameras == 1

Clamp to a maximum of 100.

Danger Points
Add up the following, then clamp to a minimum of 0:

- Crowd: 3 if playersLeft > 300, else 2 if > 150, else 1 if > 50, else 0
- Alert: 2 if alertAdj ≥ 70, else 1 if alertAdj ≥ 40, else 0
- Game: 2 if game ≥ 4, else 1 if game ≥ 2, else 0
- Cameras: +1 if cameras == 1
- Darkness: −1 if lightsOn == 0

Surcharge

- SurchargePct = 50 if dangerPoints ≥ 7, else 25 if ≥ 5, else 10 if ≥ 3, else 0
- Apply it to each item separately: itemS = (itemCost * (100 + SurchargePct)) / 100
- itemsAfterSurcharge = NeedleS + BandageS + MarbleS + RopeS

Fees

- LateFee: 250 if timeLeft < 15, else 100 if timeLeft < 45, else 0
- GuardToll: 50 + (alertAdj / 10) * 5. It becomes 0 if card == 1 and cameras == 0.
- carryCap = base (A = 40, T = 30, G = 35) + 10 if ally == 1
- WeightPenalty: min(500, 25 * max(0, packKg − carryCap))
- Wager: stake if bet != 0, else 0 (this is not counted as a fee)

Rebates and Injury Tax

- Ally Rebate: 60 if ally == 1
- Calm Rebate: 40 if injuries == 0 and stamina ≥ 70
- InjuryTax: 0 / 30 / 90 / 200 for injuries 0 / 1 / 2 / 3

Favors (exactly one, applied last; if the condition fails, FavorDiscount = 0)

- 0 = None

- 1 = INSIDER: 15% of (NeedleS + MarbleS), only if game == 2 or game == 4 (cap 500)
- 2 = HUSH: 50% of (LateFee + GuardToll), only if alertAdj ≥ 60 (cap 250)
- 3 = MEDPASS: 35% of BandageS, only if injuries ≥ 1 (cap 900)

Settlement

- gross = itemsAfterSurcharge + LateFee + GuardToll + WeightPenalty + Wager
- net = max(0, gross + InjuryTax − Ally Rebate − Calm Rebate − FavorDiscount)
- DebtFee = 80 + debt / 20 if net > cash, else 0
- Total = net + DebtFee. If Total < 200, set Total = 200.
- Fees = LateFee + GuardToll + WeightPenalty + InjuryTax + DebtFee
- CashUsed = min(cash, Total)
- Payable = Total − CashUsed

Survival Chance and Outcome

Start at 40, then:

- +min(15, ropeM / 2) if takeRope == 1
- +min(12, 2 * bandages) if takeBandages == 1
- +stamina / 4
- +8 if card == 1, else −25
- +5 if ally == 1
- −max(0, packKg − carryCap) / 2
- −5 * (SurchargePct / 10)
- −8 * injuries
- −4 * (game − 1)
- −12 if DebtFee > 0
- +10 if the item matching the current game is bought in sufficient quantity: game 2 needs takeNeedles == 1 and needles ≥ 5, game 4 needs takeMarbles == 1 and marbles ≥ 20, game 5 needs takeRope == 1 and ropeM ≥ 10. Games 1 and 3 give no bonus.

Clamp to [0..100]. Outcome = Survived if SurvivalChance ≥ 55, else Eliminated.

Prize and Final Balance

- prizePool = (456 − playersLeft) * 100
- PrizeShare = prizePool / playersLeft if Survived, else 0
- BetPayout (only if Survived, else 0):
    - bet == 1: (stake * 3) / 2
    - bet == 2: stake * 4 if SurvivalChance ≥ 75, else 0
    - bet == 0: 0
- BetPayout is the total amount returned to you, including your stake.
- FinalBalance = cash − Total + PrizeShare + BetPayout

Output Format (Exact Order & Spelling)

NeedleCost=...

BandageCost=...

MarbleCost=...

RopeCost=...

DangerPoints=...

SurchargePct=...%

LateFee=...

GuardToll=...

WeightPenalty=...

Wager=...

FavorDiscount=...

Fees=...

Total=...

CashUsed=...

Payable=...

SurvivalChance=...

Outcome=...

PrizeShare=...

BetPayout=...

FinalBalance=...

Note

- No use of if, else, or switch is allowed.
- Validate all inputs.
- No loop-based branching is allowed.
- Use integer math only.
- For favors, compute the floored percentage first, then apply the cap.
- Follow the order given exactly.
- Clamp alertAdj, DangerPoints, and SurvivalChance as specified.
- No extra inputs or hidden states are allowed.

Question 2 : Wildfire Propagation Dynamics & Telemetry Processing
[15 Marks]

Part A

Scientists at the Disaster Response and Environmental Analytics Center (DREAC) at FAST are implementing a predictive simulation model to forecast the velocity of wildfire propagation across non-uniform terrain. The spread velocity depends on several coupled environmental parameters: ambient wind speed, terrain slope angle, and vegetation fuel dryness.

Determine the estimated wildfire spread velocity ΔV (in kilometers per hour) using the model equation below:

$$\Delta V = W_0 \cdot e^{\beta \cdot U} + U_0 \cdot \sin^2(\theta_s) + \mu \cdot \ln\left(\frac{D + 1.0}{D_0}\right) + \sqrt{U \cdot \cos(\theta_s) + 1.0}$$

Where:

- ΔV: Wildfire propagation velocity (in km/h)
- U: Wind speed (in km/h)
- θ_s: Terrain slope angle (in radians)
- D: Vegetation fuel dryness index

Model Constants:

- $W_0 = 0.8, \beta = 0.12, U_0 = 15.0$
- $\mu = 3.4, D_0 = 0.5$

Note: Output the final calculated velocity ΔV formatted to exactly 4 decimal places.

Part B

To broadcast field telemetry efficiently over a low-bandwidth radio channel, the continuous velocity reading ΔV must be encoded into a single 64-bit integer telemetry key K using explicit type casting:

1. Extract the whole integer portion of ΔV and store it as V_{int}.
2. Extract the first two decimal digits (1st and 2nd decimal places) of ΔV as an integer F_1.
3. Extract the subsequent two decimal digits (3rd and 4th decimal places) of ΔV as an integer F_2.
4. Using the last four digits of your roll number (e.g., 25I-1608 → 1608), derive a roll offset factor R defined as:
   R = ((Last 4 digits of roll num) MOD 89) + 10
5. Combine the extracted components into the primary telemetry key K:

$$K = (V_{int} \cdot 10^6) + (F_1 \cdot 10^4) + (F_2 \cdot 10^2) + R$$

Part C

Receiving terminals must parse the telemetry key K to generate an operational status tag, evaluate a hardware precision flag, and compute a danger threat rating:

1. Retrieve the roll offset factor R directly from K.
2. Synthesize a 2-character ASCII status tag $(C_1 C_2)$ where:
    - C_1 is the character shifted $(V_{int}\ MOD\ 26)$ positions past 'A'.
    - C_2 is the character shifted $(R\ MOD\ 26)$ positions prior to 'Z'.
3. Reconstruct two separate estimates of the original velocity from the parsed components:
    - A single-precision floating-point value v_{float} using float arithmetic.
    - A double-precision floating-point value v_{double} using double arithmetic.

      Evaluate a precision match indicator is_exact that checks whether v_{float} equals v_{double}.
4. Calculate the integer Threat Rating incorporating a 25-point penalty if a precision mismatch occurs:

$$\text{Threat} = \text{R} + \text{Penalty} + [1.5 \cdot v_{double}]$$

Constraints & Instructions

- Input Requirements: Prompt for user inputs in the exact following order:
    1. U (Wind speed, double)
    2. θ_s (Terrain slope angle in radians, double)
    3. D (Fuel dryness index, double)
    4. rollLast4 (Last 4 digits of student roll number, long long)
- Output Format:
    1. ΔV:
    2. Telemetry Key *K*:
    3. Status Tag:
    4. Precision Match Flag:
    5. Threat Rating:
- Allowed Libraries: You are strictly limited to using functions from **<iostream>, <iomanip>, and <cmath>.**
- **Forbidden Constructs**: The use of loops (for, while, do-while), conditional branching (if, else, switch), or conditional operators (? :) is strictly prohibited and will result in **zero** marks. All computations and logical flags must be evaluated purely through arithmetic expressions and type casting.

Question 3 : Women and Children First [20 Marks]

April 14, 1912. 11:40 PM. The iceberg has struck.

Water is pouring into the lower decks, and two crew members are trying to reach the **Lifeboat Deck (Deck 6)** from the flooded **Boiler Room (Deck 0)** before the corridors seal.

Officer Murdoch doesn't waste time deliberating — once he commits to a corridor, he follows it as far as it goes, and only turns back when he's truly stuck.

Stewardess Mary is more careful — she checks every nearby corridor before venturing further from where she started, making sure she isn't missing a closer way out.

Simulate both of them for exactly **6 moves each**, using **only cout and switch-case**. **No if-else, no ternary, no loops, no arrays, no functions.**

Decks

```
0 = Boiler Room      4 = Promenade Deck
1 = Cargo Hold       5 = Bridge
2 = Third Class      6 = Lifeboat Deck
3 = Dining Saloon    7 = Crow's Nest
```

Corridors (two-way unless marked)

```
0-1, 0-2, 1-3, 1-4, 2-5, 2-6, 3-7, 4-7, 5-6
5 → 7 (a narrow maintenance ladder — climbable only from the Bridge up to the Crow's Nest, not back down)
```

Setup

Declare plain named variables per person, one per deck (16 total): murVisitedBoilerRoom, murVisitedCargoHold, murVisitedThirdClass, murVisitedDiningSaloon, murVisitedPromenadeDeck, murVisitedBridge, murVisitedLifeboatDeck, murVisitedCrowsNest, and the same eight for mary… All start as 0 (unvisited). Since only switch-case is allowed, represent every condition as an integer value you can switch on.

Track murCurrent and maryCurrent, both starting at 0.

Rules — apply in this exact order, at every move, for each person separately

1 **Flooding:** switch on the current deck — if it is Third Class (case 2), switch again on whether the Bridge has been visited; if not, print `WaterRising`, send that person straight to the Bridge this move (skip rules 2 and 3 for this move), and mark the Bridge visited.

2 **Find corridors:** use switch(current) to list that deck's legal connections from the corridor list above. Remember — the Crow's Nest ladder only goes up, not back down to the Bridge.

3 **Choose the next deck:**

- **Murdoch's rule:** always take the smallest-numbered unvisited connecting deck.

Encode this by switching on the current deck, and inside each case, nest further switches (e.g. on a visited-status code) to pick the smallest unvisited option in order. If every connecting deck is already visited, print `TurnBack` and switch your way to the smallest-numbered visited deck that still leads somewhere new. If nothing qualifies, print `Trapped` and stay put for the rest of his moves.

- **Mary's rule:** she won't move further away until she's checked everything close by first — finishing every deck one corridor away from the Boiler Room before trying anything two corridors away, and so on. Work this out by hand for each move and encode the decision as nested switches (e.g. switch on the current deck, then switch on which neighbours are unvisited).

4 **Reached safety:** switch(current) — if it is the Lifeboat Deck (case 6), print `Rescued` and freeze that person for all remaining moves (use a separate frozen flag, checked via switch at the start of each later move).

5 **Door seal glitch:** switch(current) — if it is the Crow's Nest (case 7), switch again on whether the Dining Saloon was visited; if yes, print `DoorSealed` and mark the Dining Saloon unvisited again.

Update the visited flag for the new deck and update current before moving to the next move.

Output Format

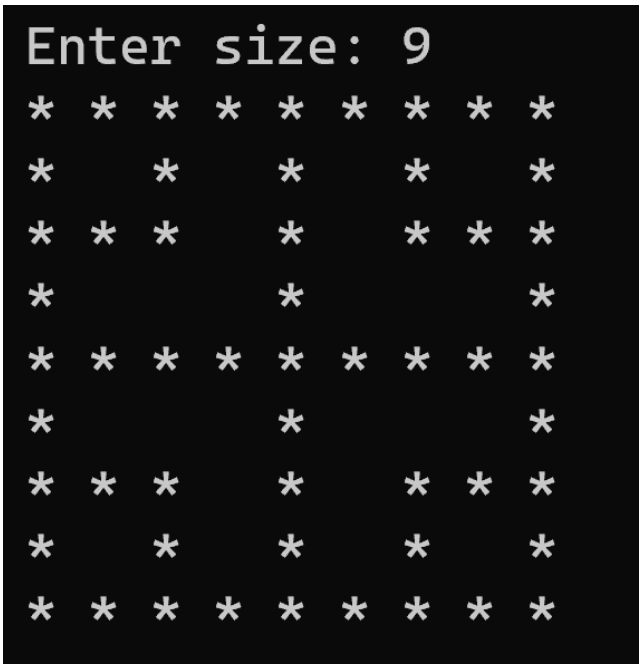
```
Murdoch path: <deck names separated by ->>
Mary path: <deck names separated by ->>
```

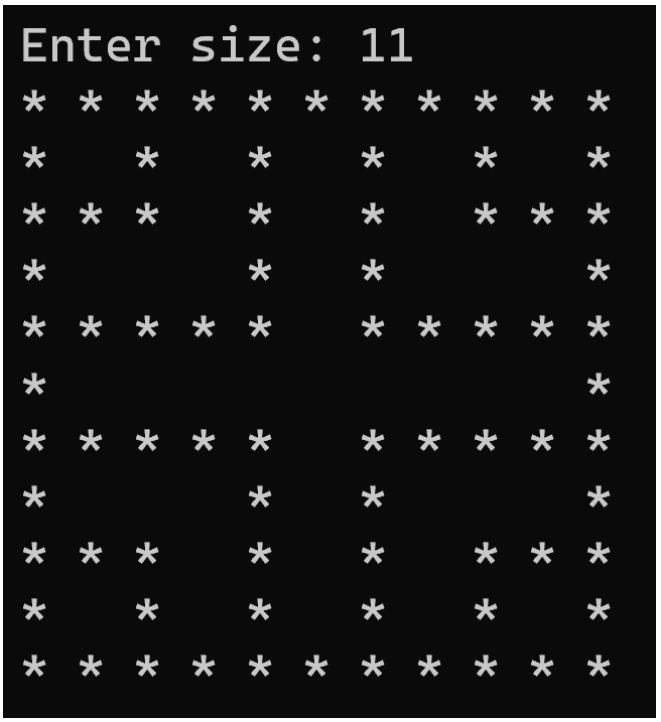
Constraints

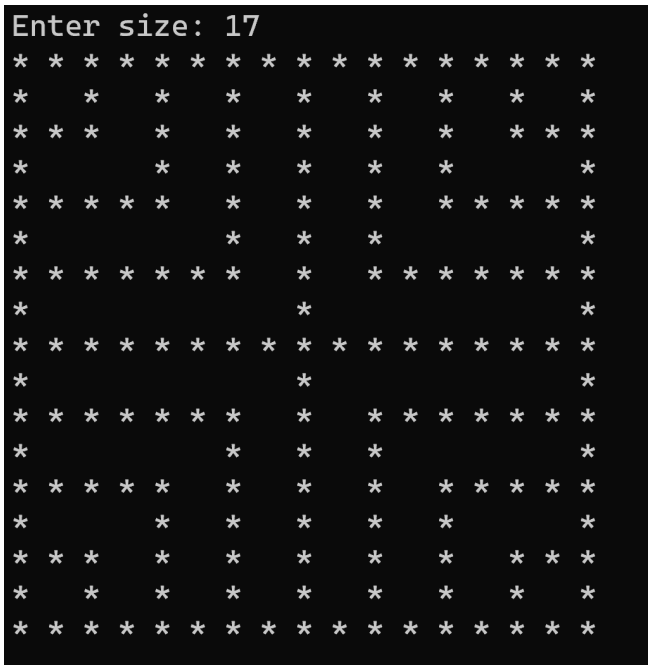
- Allowed: `cout`, `switch-case`, and plain `int` variables only.
- Write out all 6 moves individually for each person (12 blocks total in your code).
- Since switch only compares one integer value at a time for exact equality, every comparison (visited/unvisited, smallest-numbered, etc.) must be worked out using nested switch statements on int variables.

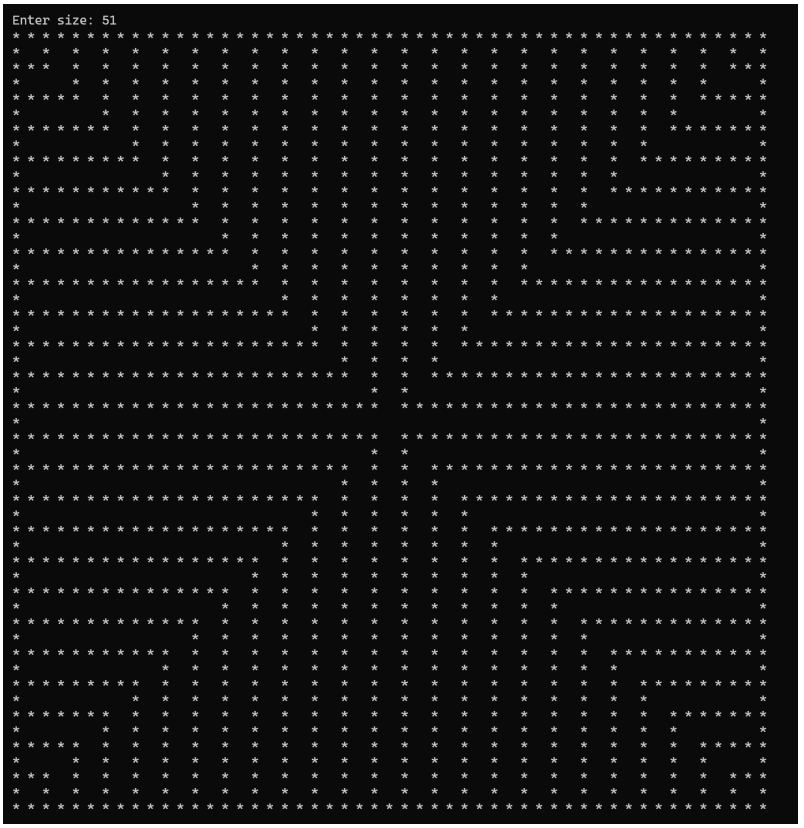
Question 4 : Square Trouble [10 Marks]

Write a program that prints the following pattern based on the value of the input.





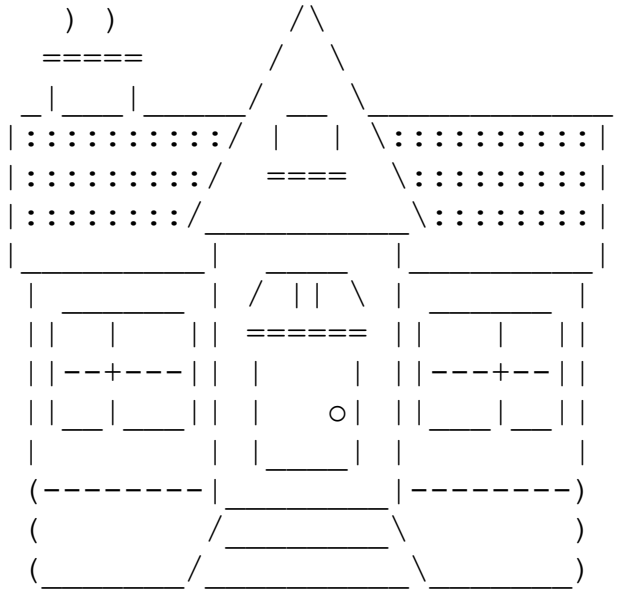




Question 5 : Bob The Printer [15 Marks]

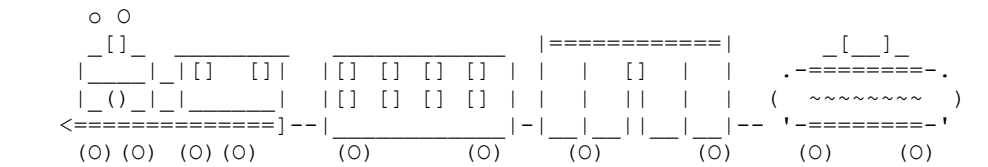
Forget 3d printed houses, the new fad is all about two dimensions. You have been hired as the chief architect for Mamdanistan, a new digital landscape for which the audience has yet to be decided. You have to construct 2 designs, and modify them relevantly according to the constraints, which will be provided dynamically during construction.

Part A - The Humble Homeowner



Defining features are one central door and attic. ***Inputs*** *: a) Width and height of the entire house, b) width and height of the door, c) height of the attic from base to roof (6 in this example), d) number of chimneys, e) height of walkway (the part below the door, 2 in this example). Chimneys and windows will always be as big as the one in this example, and if a wider house input is selected then additional windows must be made according to the width of the house. Make sure to validate all inputs.*

Part B - Thomas the Tank and some more Engine



A city needs a proper locomotive system, and this prototype is made up of 4 distinct types of compartments. Left to Right : Engine (1), Cabin (2), Cargo (3), VIP (4). Input will be given as a 4 digit integer, and the order will determine the locomotive's design. The example given had input of 1234. Sizes are exactly as specified.

Question 6 : Verity [20 Marks]

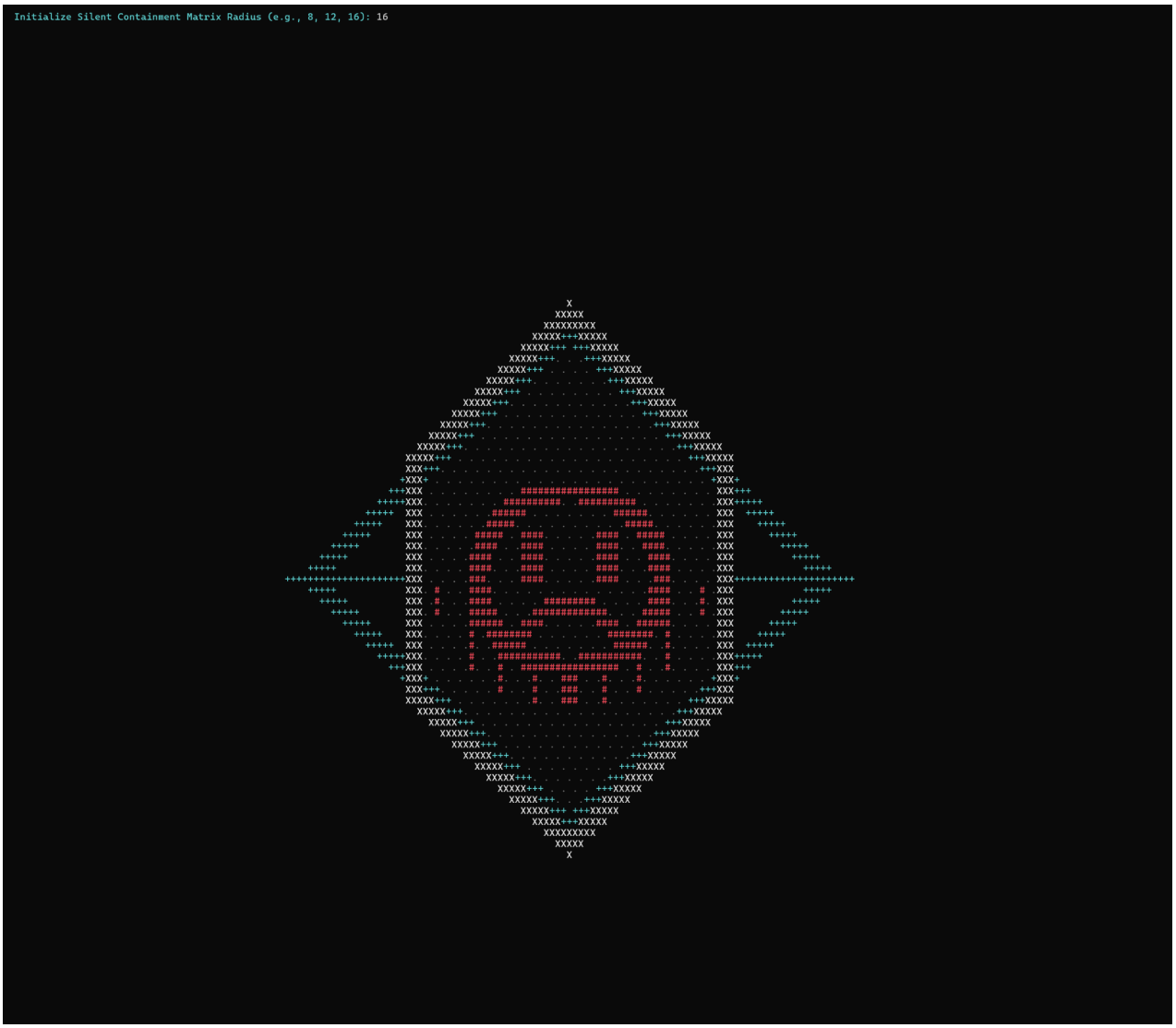
Write a program using nested loops and conditional statements to print the given ASCII art pattern exactly as shown.

The pattern consists of a large diamond-shaped outer structure made using characters, with a circular Red John–style smiley face in the center.

Your program should reproduce the complete pattern with the correct spacing, alignment, symbols, and colors/characters as shown in the sample output.

The size of the pattern should be based on the given input value (e.g., 24), allowing the design to be generated systematically rather than printing each line individually.

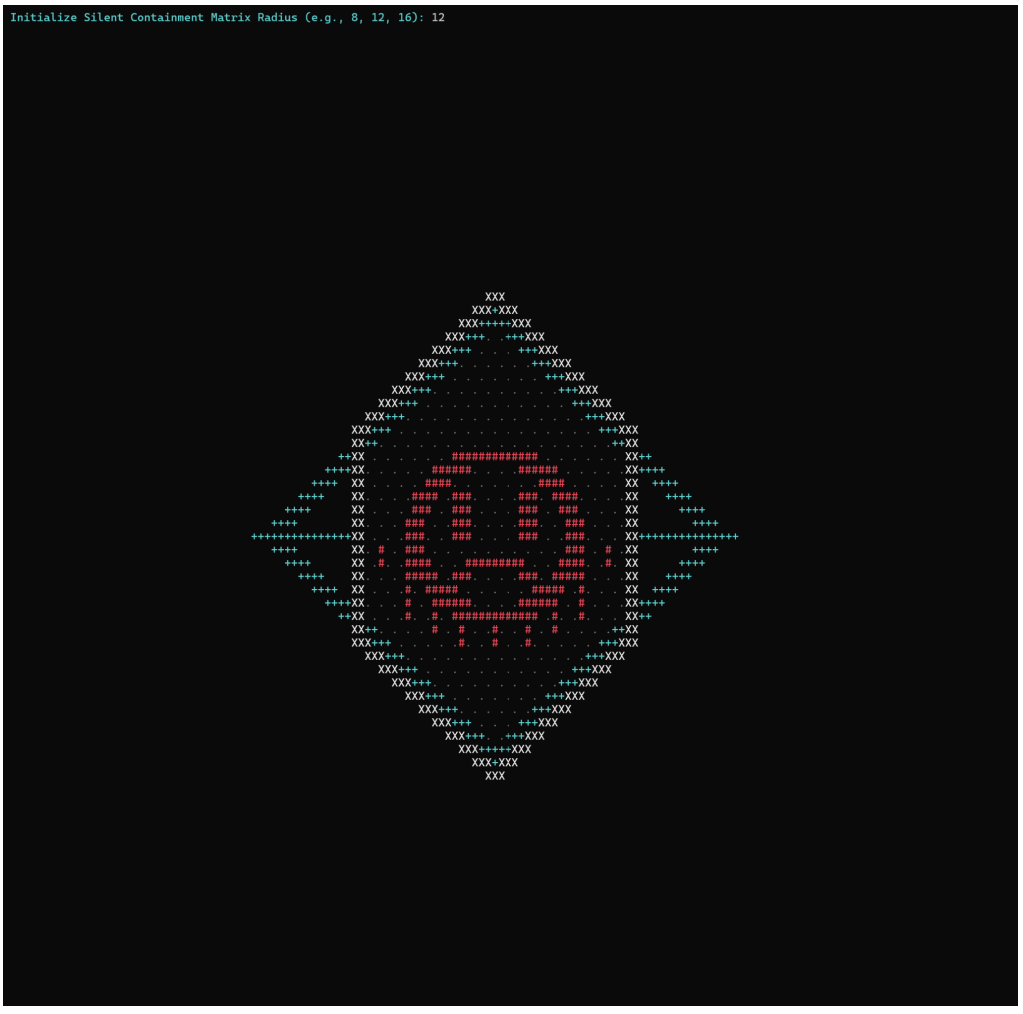
INPUT : 16



INPUT : 24



INPUT : 12



Question 7 : Calcu-later [25 Marks]

You have a Calculus exam tomorrow, and you cannot find your calculator! Fortunately, the exam will only consist of calculations with operands of up to 6 bits, which can be both signed AND unsigned. You have the night to construct a calculator that takes inputs of both operands, specify their signage, and then display each operands' decimal representation, and the results of their addition, subtraction (both A-B and B-A), multiplication, and division.

No bitwise operators are allowed. You must write your own code to simulate them on integers that have only 1s and 0s as digits. So in your code, each bit is represented as one integer digit. For example, 001101 may be 13 in binary, within your code it will be inputted and stored as one thousand one hundred and one.

Your calculator must detect overflow (and underflow) in addition and subtraction. Whenever it occurs, the answer must be displayed using 7 bits instead of 6, along with a message saying that it occurred.

All your outputs must be displayed as both binary and decimal representation, and you cannot use print loops to display numbers bit by bit, all displaying of numbers must be done via cout << variablename.

All calculations must be done bit/digit by bit/digit, you are not allowed to directly store them in decimal values and calculate them and reconvert to binary. ZERO will be given in that case.

Sample Input / Output :-

Input A : 101000
Input B : 1100
Are these numbers signed or unsigned? Y/N : Y
… Calculating …
A : -24
B : 12
A + B = 110100 = -12 **no overflow, so 6 bits**
A - B = 1011100 = -36 **overflow detected, answer given in 7 bits**
B - A = 0100100 = 36 **overflow detected, answer given in 7 bits**
A * B = 1110 1110 0000 = -288 **must be 12 bits/digits long, sign/unsigned according to input**
A / B = 111110 = -2 **give only the quotient**
A % B = 000000 = 0 **give only the remainder, inheriting sign of dividend**

Question 8 : Pac-a-Man [30 Marks]

You've all heard of the classic Pac-Man arcade game, but there is now a twist. Our brave hero is now trapped in the Cartesian plane (of **20** width and **10** height) and is being chased by his 4 enemies : **Blinky**, **Pinky**, **Inky**, and **Clyde**. However, all 4 ghosts have had their movements constrained to certain ley lines by a magical spell, and can only travel along paths fulfilling certain conditions.

Blinky : can only travel along the horizontal line of height 3 from the top, left to right. When he reaches the end, he teleports down to a height of 3 from the bottom, and must travel right to left. When he reaches the end of his bottom journey, he restarts at the left side, height 3 from the top. He moves exactly one block every iteration.

Pinky : Pinky is constrained to only move one unit in Pac-Man's direction every turn, and the decision of moving horizontally or vertically depends on which direction will bring her closer. For example, if she is 2 blocks to the left and 3 blocks above Pac-Man, she will move right instead of down. In case of a tie, she will move one block in a diagonal direction.

Inky : The shyest of the bunch. He will remain stationary, unless he is the closest in Euclidean distance of the ghosts to Pac-Man. In that case, he will teleport exactly one block away from Pac-Man into a free space, with the decision starting clockwise from the block above Pac-Man.

Clyde : Clyde is secretly a double agent, and is on Pac-Man's side. He is bound by the spell to move exactly as Pac-Man moves every iteration, however he cannot move out of the boundary of the plane, so he can be forced into a boundary so that Pac-Man can approach him. Once Pac-Man and Clyde are in the same cell, Clyde will activate a siren that calls the Ghostbusters, saving Pac-Man and ending the game.

If any of the other 3 ghosts catch Pac-Man before Clyde does, Pac-Man loses and the game ends.
The positions of all beings at the start of the game are randomized.
The order of events in every iteration are as follows :-

- Pac-Man's input is taken, WASD for up left down right, and his new position is updated.
- Each ghost's logic is checked and movement updated, in the order of Clyde, Inky, Pinky, and Blinky.
- After every ghost's movement, a check is done to see if the game has been lost/won.
- Finally, the board is drawn, with * for free space, @ for pacman, B P I C for the ghosts.

You are not allowed to use if-else in this question. All decisions must be done by switch cases or ternary operators. Arrays are strictly not allowed.

Good luck, you'll need it!